
Session 24 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rath**

College: **Zeal Polytechnic, Narhe, Pune**

Branch: **Artificial Intelligence and Machine Learning (AIML)**

Domain: **AIML (Artificial Intelligence and Machine Learning)**

Github Main Repo Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments

Github File Link :

Regression : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session24/Assignment%2024%20-%20Regression.ipynb

Classification : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session24/Assignment%2024%20-%20Classification.ipynb

Repo created for Deployment :

GitHub Repository: https://github.com/vaishnavi-rathiii/AI-Prediction-System_Streamlit

Deployed Streamlit App: <https://ai-prediction-systemapp-nxckwqvf8rsi42u99nixk.streamlit.app/>

Code & Output

Regression :

Dataset Used: – Kaggle - <https://www.kaggle.com/datasets/mirichoi0218/insurance>

Q1. Dataset Selection & Preprocessing

1. Import Required Libraries

```
[2]: '''  
      Import Required Libraries  
      '''  
  
      import pandas as pd  
      import numpy as np  
  
      import matplotlib.pyplot as plt  
      import seaborn as sns  
  
      from sklearn.preprocessing import StandardScaler  
      from sklearn.model_selection import train_test_split  
      from sklearn.linear_model import LinearRegression  
  
      from sklearn.metrics import (  
          mean_absolute_error,  
          mean_squared_error,  
          r2_score  
      )  
  
      import joblib
```

2. Load Dataset

```
[3]: '''
Load Dataset
'''

# Load Dataset
df = pd.read_csv("insurance.csv")

# Display first 5 rows
print("First 5 Rows:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

print("\nDataset Information:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Rows:")
print(df.duplicated().sum())
```

First 5 Rows:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

Dataset Shape:
(1338, 7)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
Column Non-Null Count Dtype
--- ---
0 age 1338 non-null int64
1 sex 1338 non-null object
2 bmi 1338 non-null float64
3 children 1338 non-null int64
4 smoker 1338 non-null object
5 region 1338 non-null object
6 charges 1338 non-null float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
None

Missing Values:

age	0
sex	0
bmi	0
children	0
smoker	0
region	0
charges	0

dtype: int64

Duplicate Rows:

1

3. Data Cleaning & Preprocessing

```
[4]: '''
Data Cleaning & Preprocessing
'''

# Remove duplicate rows
df.drop_duplicates(inplace=True)

print("Dataset Shape:", df.shape)

# Separate Features and Target
X = df.drop("charges", axis=1)
y = df["charges"]

print("\nShape of X:", X.shape)
print("Shape of y:", y.shape)

Dataset Shape: (1337, 7)

Shape of X: (1337, 6)
Shape of y: (1337,)
```

4. Exploratory Data Analysis (EDA)

```
[5]: '''
Exploratory Data Analysis
'''

# Charges Distribution
plt.figure(figsize=(6,4))
sns.histplot(df["charges"], kde=True)
plt.title("Insurance Charges Distribution")
plt.show()

# Age Distribution
plt.figure(figsize=(6,4))
sns.histplot(df["age"], kde=True)
plt.title("Age Distribution")
plt.show()

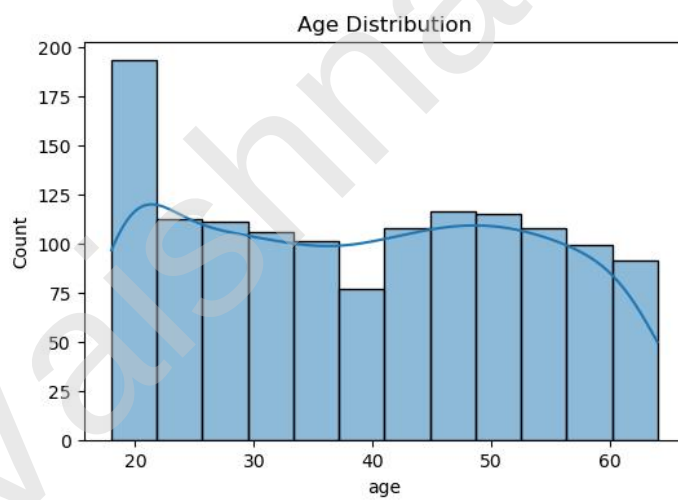
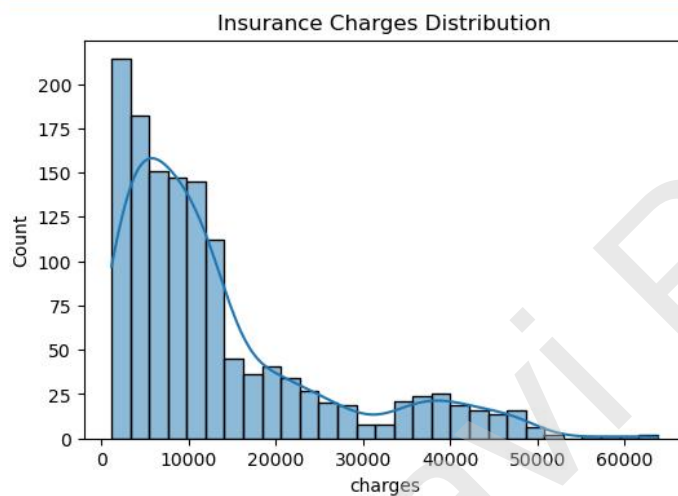
# Charges by Smoker
plt.figure(figsize=(6,4))
sns.boxplot(x="smoker", y="charges", data=df)
plt.title("Insurance Charges by Smoker")
plt.show()

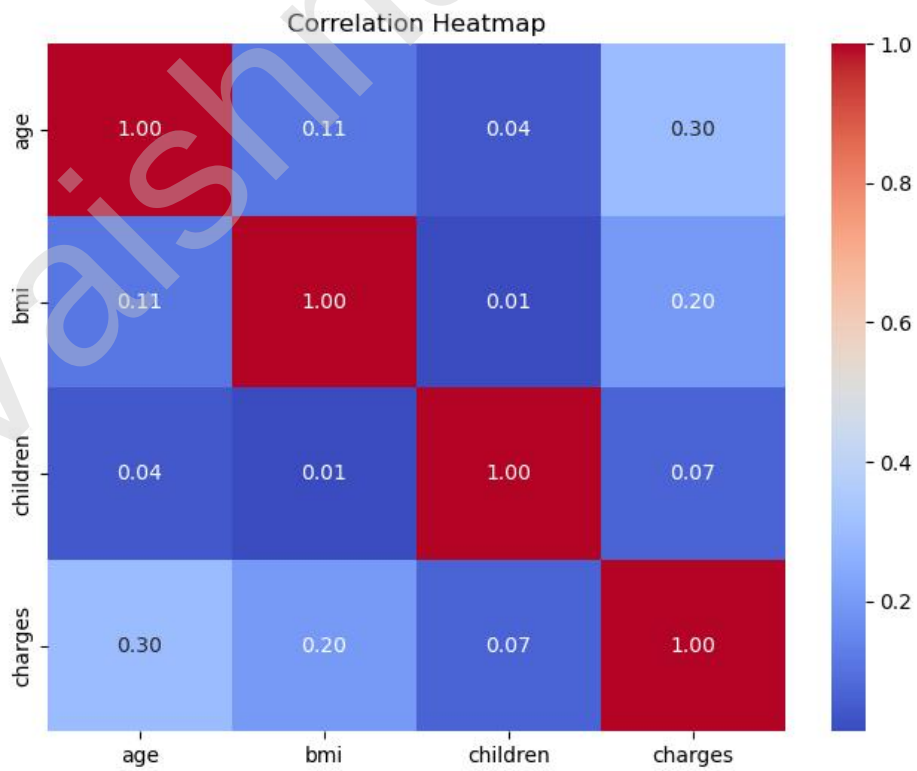
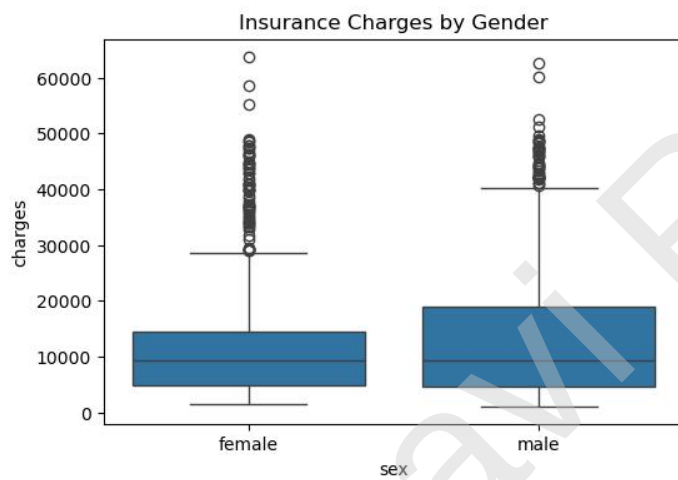
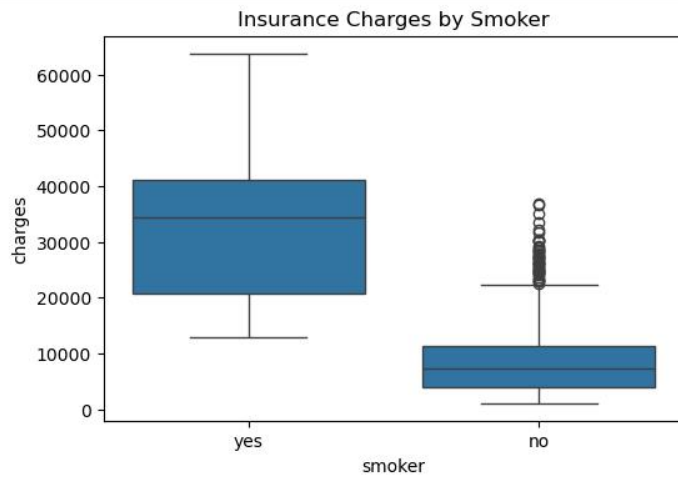
# Charges by Gender
plt.figure(figsize=(6,4))
sns.boxplot(x="sex", y="charges", data=df)
plt.title("Insurance Charges by Gender")
plt.show()
```

```
# Correlation Heatmap
plt.figure(figsize=(8,6))

sns.heatmap(
    df.select_dtypes(include=np.number).corr(),
    annot=True,
    fmt=".2f",
    cmap="coolwarm"
)

plt.title("Correlation Heatmap")
plt.show()
```





5. Feature Encoding & Scaling

```
[7]: '''  
      Encoding & Feature Scaling  
      '''  
  
      # One-Hot Encoding  
      X = pd.get_dummies(X, drop_first=True)  
  
      # Save Column Names  
      columns = X.columns  
  
      # Feature Scaling  
      scaler = StandardScaler()  
  
      X = scaler.fit_transform(X)  
  
      print("Encoding and Scaling Completed Successfully.")  
      Encoding and Scaling Completed Successfully.
```

6. Train-Test Split

```
[8]: '''  
      Train-Test Split  
      '''  
  
      X_train, X_test, y_train, y_test = train_test_split(  
          X,  
          y,  
          test_size=0.20,  
          random_state=42  
      )  
  
      print("X_train:", X_train.shape)  
      print("X_test :", X_test.shape)  
      print("y_train:", y_train.shape)  
      print("y_test :", y_test.shape)  
  
      X_train: (1069, 8)  
      X_test : (268, 8)  
      y_train: (1069,)  
      y_test : (268,)
```

Q3. (Regression Algorithms)

7. Linear Regression Model Training

```
Linear Regression Model Training
'''
|
from sklearn.linear_model import LinearRegression

# Create Model
lr_model = LinearRegression()

# Train Model
lr_model.fit(X_train, y_train)

print("Linear Regression Model Trained Successfully.")

Linear Regression Model Trained Successfully.

[49]: '''
Model Prediction
'''

y_pred_lr = lr_model.predict(X_test)

print("First 10 Actual Values:")
print(y_test.iloc[:10].values)

print("\nFirst 10 Predicted Values:")
print(y_pred_lr[:10])

First 10 Actual Values:
[ 8688.85885  5708.867  11436.73815  38746.3551  4463.2051  9304.7019
 38511.6283  2150.469  7345.7266  10264.4421 ]

First 10 Predicted Values:
[ 8143.69388412  5737.11568259  14369.31487618  31745.51363586
 8962.38665706  13149.72235307  30446.76067941  1453.28881259
 10633.01840233  11318.94379361]

[50]: '''
Model Evaluation
'''

from sklearn.metrics import r2_score

r2_lr = r2_score(y_test, y_pred_lr)

print("R2 Score :", round(r2_lr,4))

R2 Score : 0.8069
```


8. Decision Tree Regressor Model Training

```
[51]: '''  
Decision Tree Regressor Model Training  
'''  
  
from sklearn.tree import DecisionTreeRegressor  
  
# Create Model  
dt_model = DecisionTreeRegressor(random_state=42)  
  
# Train Model  
dt_model.fit(X_train, y_train)  
  
print("Decision Tree Regressor Trained Successfully.")
```

Decision Tree Regressor Trained Successfully.

```
[52]: '''  
Model Prediction  
'''  
  
y_pred_dt = dt_model.predict(X_test)  
  
print("First 10 Actual Values:")  
print(y_test.iloc[:10].values)  
  
print("\nFirst 10 Predicted Values:")  
print(y_pred_dt[:10])
```

First 10 Actual Values:
[8688.85885 5708.867 11436.73815 38746.3551 4463.2051 9304.7019
 38511.6283 2150.469 7345.7266 10264.4421]

First 10 Predicted Values:
[9361.3268 4877.98105 10848.1343 44585.45587 4673.3922 8978.1851
 36219.40545 2154.361 19144.57652 10269.46]

```
[53]: '''  
Model Evaluation  
'''  
  
r2_dt = r2_score(y_test, y_pred_dt)  
  
print("R2 Score :", round(r2_dt,4))
```

R² Score : 0.8137

9. Support Vector Regressor (SVR) Model Training

```
[54]: '''  
Support Vector Regressor (SVR) Model Training  
'''  
  
from sklearn.svm import SVR  
  
# Create Model  
svr_model = SVR(kernel="rbf")  
  
# Train Model  
svr_model.fit(X_train, y_train)  
  
print("SVR Model Trained Successfully.")  
  
SVR Model Trained Successfully.
```

```
[55]: '''  
Model Prediction  
'''  
  
y_pred_svr = svr_model.predict(X_test)  
  
print("First 10 Actual Values:")  
print(y_test.iloc[:10].values)  
  
print("\nFirst 10 Predicted Values:")  
print(y_pred_svr[:10])  
  
First 10 Actual Values:  
[ 8688.85885  5708.867  11436.73815  38746.3551  4463.2051  9304.7019  
 38511.6283  2150.469  7345.7266  10264.4421 ]  
  
First 10 Predicted Values:  
[9286.5177216  9275.79595528  9306.19330745  9320.65864281  9257.12710605  
 9291.3572724  9320.54206512  9239.86823456  9274.8216007  9290.72032035]
```

```
[56]: '''  
Model Evaluation  
'''  
  
r2_svr = r2_score(y_test, y_pred_svr)  
  
print("R2 Score :", round(r2_svr,4))  
  
R2 Score : -0.1312
```

10. K-Nearest Neighbors Regressor Model Training

```
[57]: '''  
      K-Nearest Neighbors Regressor Model Training  
      '''  
  
      from sklearn.neighbors import KNeighborsRegressor  
  
      scores = {}  
  
      for k in [3,5,7]:  
  
          knr = KNeighborsRegressor(n_neighbors=k)  
  
          knr.fit(X_train, y_train)  
  
          pred = knr.predict(X_test)  
  
          scores[k] = r2_score(y_test, pred)  
  
          print(f"K = {k} --> R2 Score = {scores[k]:.4f}")  
  
      K = 3 --> R2 Score = 0.8501  
      K = 5 --> R2 Score = 0.8371  
      K = 7 --> R2 Score = 0.8287  
  
[58]: '''  
      Select Best K Value  
      '''  
  
      best_k = max(scores, key=scores.get)  
  
      print("Best K Value :", best_k)  
  
      Best K Value : 3
```

```
[59]: '''
Final KNN Regressor Model Training
'''

knn_model = KNeighborsRegressor(n_neighbors=best_k)

knn_model.fit(X_train, y_train)

print("Final KNN Regressor Trained Successfully.")

Final KNN Regressor Trained Successfully.
```

```
[60]: '''
Model Prediction
'''

y_pred_knn = knn_model.predict(X_test)

print("First 10 Actual Values:")
print(y_test.iloc[:10].values)

print("\nFirst 10 Predicted Values:")
print(y_pred_knn[:10])

First 10 Actual Values:
[ 8688.85885  5708.867   11436.73815  38746.3551   4463.2051   9304.7019
  38511.6283   2150.469    7345.7266  10264.4421 ]

First 10 Predicted Values:
[ 8873.87071667  9798.03057667 11152.81133333 31417.99958
 4618.56566667  8162.45386667 48038.82379333 2067.60766667
 7551.88813333 10163.44633333]
```

```
[61]: '''
Model Evaluation
'''

r2_knn = r2_score(y_test, y_pred_knn)

print("R2 Score :", round(r2_knn,4))

R2 Score : 0.8501
```

Q4. Best Model Selection & Saving

11. Performance Comparison of Models

```
[62]: '''  
Performance Comparison of Models  
'''  
  
comparison = pd.DataFrame({  
    "Algorithm": [  
        "Linear Regression",  
        "Decision Tree Regressor",  
        "Support Vector Regressor",  
        f"KNN Regressor (K={best_k})"  
    ],  
    "R2 Score": [  
        r2_lr,  
        r2_dt,  
        r2_svr,  
        r2_knn  
    ]  
})  
  
comparison = comparison.round(4)  
  
print("Model Performance Comparison:\n")  
print(comparison)
```

Model Performance Comparison:

	Algorithm	R ² Score
0	Linear Regression	0.8069
1	Decision Tree Regressor	0.8137
2	Support Vector Regressor	-0.1312
3	KNN Regressor (K=3)	0.8501

```
[63]: '''
Best Performing Algorithm
'''

best_algorithm = comparison.loc[
    comparison["R2 Score"].idxmax(),
    "Algorithm"
]

print("Best Performing Algorithm:", best_algorithm)

print("\nConclusion:")
print(f"{best_algorithm} achieved the highest R2 Score and performed best on the Medical Insurance dataset.")

Best Performing Algorithm: KNN Regressor (K=3)

Conclusion:
KNN Regressor (K=3) achieved the highest R2 Score and performed best on the Medical Insurance dataset.

[64]: '''
Best Regression Model
'''

best_regression_model = comparison.loc[
    comparison["R2 Score"].idxmax(),
    "Algorithm"
]

print("Best Regression Model:", best_regression_model)

Best Regression Model: KNN Regressor (K=3)
```

12. Save Best Regression Model

```
[65]: '''
Save Best Regression Model
'''

if best_regression_model == "Linear Regression":
    best_reg_model = lr_model

elif best_regression_model == "Decision Tree Regressor":
    best_reg_model = dt_model

elif best_regression_model == "Support Vector Regressor":
    best_reg_model = svr_model

else:
    best_reg_model = knn_model

joblib.dump(best_reg_model, "best_regression_model.pkl")
joblib.dump(scaler, "regression_scaler.pkl")
joblib.dump(columns, "regression_columns.pkl")

print("Best Regression Model Saved Successfully.")

Best Regression Model Saved Successfully.
```

Code & Output

Classification :

Dataset Used: – Kaggle - <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

Q1. Dataset Selection & Preprocessing

1) Import Required Libraries

```
[34]: '''
      Import Required Libraries
      '''

      # Data Handling
      import pandas as pd
      import numpy as np

      # Data Visualization
      import matplotlib.pyplot as plt
      import seaborn as sns

      # Preprocessing
      from sklearn.preprocessing import LabelEncoder, StandardScaler
      from sklearn.model_selection import train_test_split

      # Machine Learning Algorithms
      from sklearn.linear_model import LogisticRegression
      from sklearn.neighbors import KNeighborsClassifier
      from sklearn.naive_bayes import GaussianNB

      # Evaluation Metrics
      from sklearn.metrics import (
          accuracy_score,
          precision_score,
          recall_score,
          f1_score,
          confusion_matrix,
          classification_report
      )

      # Save Model
      import joblib
```


2) Load Dataset

```
[35]: '''
      Load Dataset
      '''

      # Load dataset
      df = pd.read_csv("Employee-Attrition.csv")

      # Display first 5 rows
      print("First 5 Rows:")
      print(df.head())

      print("\nDataset Shape:")
      print(df.shape)

      print("\nDataset Information:")
      print(df.info())

      print("\nMissing Values:")
      print(df.isnull().sum())

      print("\nDuplicate Rows:")
      print(df.duplicated().sum())
```

First 5 Rows:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	\
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	\
0	0.11840	0.27760	0.3001	0.14710	
1	0.08474	0.07864	0.0869	0.07017	
2	0.10960	0.15990	0.1974	0.12790	
3	0.14250	0.28390	0.2414	0.10520	
4	0.10030	0.13280	0.1980	0.10430	

	... texture_worst	perimeter_worst	area_worst	smoothness_worst	\
0	... 17.33	184.60	2019.0	0.1622	
1	... 23.41	158.80	1956.0	0.1238	
2	... 25.53	152.50	1709.0	0.1444	
3	... 26.50	98.87	567.7	0.2098	
4	... 16.67	152.20	1575.0	0.1374	

	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	\
0	0.6656	0.7119	0.2654	0.4601	
1	0.1866	0.2416	0.1860	0.2750	
2	0.4245	0.4504	0.2430	0.3613	
3	0.8663	0.6869	0.2575	0.6638	
4	0.2050	0.4000	0.1625	0.2364	

	fractal_dimension_worst	Unnamed: 32
0	0.11890	NaN
1	0.08902	NaN
2	0.08758	NaN
3	0.17300	NaN
4	0.07678	NaN

[5 rows x 33 columns]

Dataset Shape:
(569, 33)

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 569 entries, 0 to 568

Data columns (total 33 columns):

#	Column	Non-Null Count	Dtype
0	id	569 non-null	int64
1	diagnosis	569 non-null	object
2	radius_mean	569 non-null	float64
3	texture_mean	569 non-null	float64
4	perimeter_mean	569 non-null	float64
5	area_mean	569 non-null	float64
6	smoothness_mean	569 non-null	float64
7	compactness_mean	569 non-null	float64
8	concavity_mean	569 non-null	float64
9	concave points_mean	569 non-null	float64
10	symmetry_mean	569 non-null	float64
11	fractal_dimension_mean	569 non-null	float64
12	radius_se	569 non-null	float64
13	texture_se	569 non-null	float64
14	perimeter_se	569 non-null	float64
15	area_se	569 non-null	float64
16	smoothness_se	569 non-null	float64
17	compactness_se	569 non-null	float64
18	concavity_se	569 non-null	float64
19	concave points_se	569 non-null	float64
20	symmetry_se	569 non-null	float64
21	fractal_dimension_se	569 non-null	float64
22	radius_worst	569 non-null	float64
23	texture_worst	569 non-null	float64
24	perimeter_worst	569 non-null	float64
25	area_worst	569 non-null	float64
26	smoothness_worst	569 non-null	float64
27	compactness_worst	569 non-null	float64
28	concavity_worst	569 non-null	float64
29	concave points_worst	569 non-null	float64
30	symmetry_worst	569 non-null	float64
31	fractal_dimension_worst	569 non-null	float64
32	Unnamed: 32	0 non-null	float64

dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
None

Missing Values:

id	0
diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0
fractal_dimension_se	0
radius_worst	0
texture_worst	0
perimeter_worst	0
area_worst	0

smoothness_worst	0
compactness_worst	0
concavity_worst	0
concave points_worst	0
symmetry_worst	0
fractal_dimension_worst	0
Unnamed: 32	569
dtype: int64	

Duplicate Rows:

0

3) Data Cleaning & Preprocessing

```
[36]: '''
Data Cleaning & Preprocessing
'''

# Drop unnecessary columns
df.drop(["id", "Unnamed: 32"], axis=1, inplace=True)

# Check duplicate rows
print("Duplicate Rows:", df.duplicated().sum())

# Remove duplicate rows
df.drop_duplicates(inplace=True)

# Encode Target Variable
# M = 1 (Malignant)
# B = 0 (Benign)
df["diagnosis"] = df["diagnosis"].map({
    "M": 1,
    "B": 0
})
```

```
# Separate Features and Target
X = df.drop("diagnosis", axis=1)
y = df["diagnosis"]
```

```
print("\nShape of X:", X.shape)
print("Shape of y:", y.shape)
```

```
print("\nFirst 5 Rows:")
print(X.head())
```

Duplicate Rows: 0

Shape of X: (569, 30)
Shape of y: (569,)

First 5 Rows:

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	

	compactness_mean	concavity_mean	concave points_mean	symmetry_mean	\
0	0.27760	0.3001	0.14710	0.2419	
1	0.07864	0.0869	0.07017	0.1812	
2	0.15990	0.1974	0.12790	0.2069	
3	0.28390	0.2414	0.10520	0.2597	
4	0.13280	0.1980	0.10430	0.1809	

	fractal_dimension_mean	...	radius_worst	texture_worst	perimeter_worst	\
0	0.07871	...	25.38	17.33	184.60	
1	0.05667	...	24.99	23.41	158.80	
2	0.05999	...	23.57	25.53	152.50	
3	0.09744	...	14.91	26.50	98.87	
4	0.05883	...	22.54	16.67	152.20	

	area_worst	smoothness_worst	compactness_worst	concavity_worst	\
0	2019.0	0.1622	0.6656	0.7119	
1	1956.0	0.1238	0.1866	0.2416	
2	1709.0	0.1444	0.4245	0.4504	
3	567.7	0.2098	0.8663	0.6869	
4	1575.0	0.1374	0.2050	0.4000	

	concave points_worst	symmetry_worst	fractal_dimension_worst
0	0.2654	0.4601	0.11890
1	0.1860	0.2750	0.08902
2	0.2430	0.3613	0.08758
3	0.2575	0.6638	0.17300
4	0.1625	0.2364	0.07678

[5 rows x 30 columns]

4) Exploratory Data Analysis (EDA)

```
[37]: '''
Exploratory Data Analysis
'''

# Diagnosis Distribution
plt.figure(figsize=(5,4))
sns.countplot(x="diagnosis", data=df)
plt.title("Diagnosis Distribution")
plt.xlabel("Diagnosis (0=Benign, 1=Malignant)")
plt.ylabel("Count")
plt.show()

# Pie Chart
plt.figure(figsize=(6,6))
df["diagnosis"].value_counts().plot(
    kind="pie",
    autopct="%1.1f%",
    labels=["Benign", "Malignant"],
    startangle=90
)
plt.title("Diagnosis Distribution")
plt.ylabel("")
plt.show()

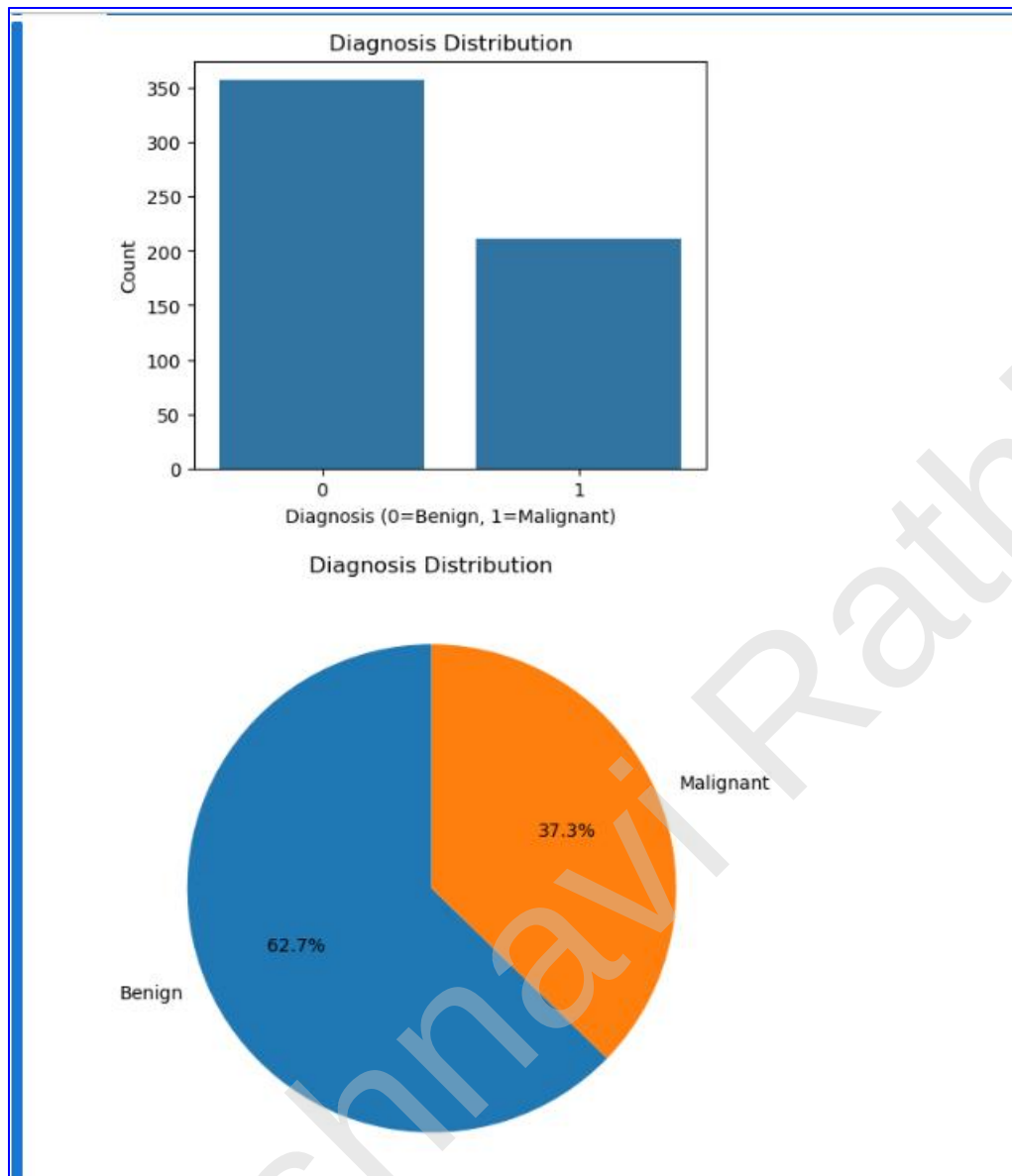
# Radius Mean Distribution
plt.figure(figsize=(7,4))
sns.histplot(df["radius_mean"], bins=20, kde=True)
plt.title("Radius Mean Distribution")
plt.show()
```

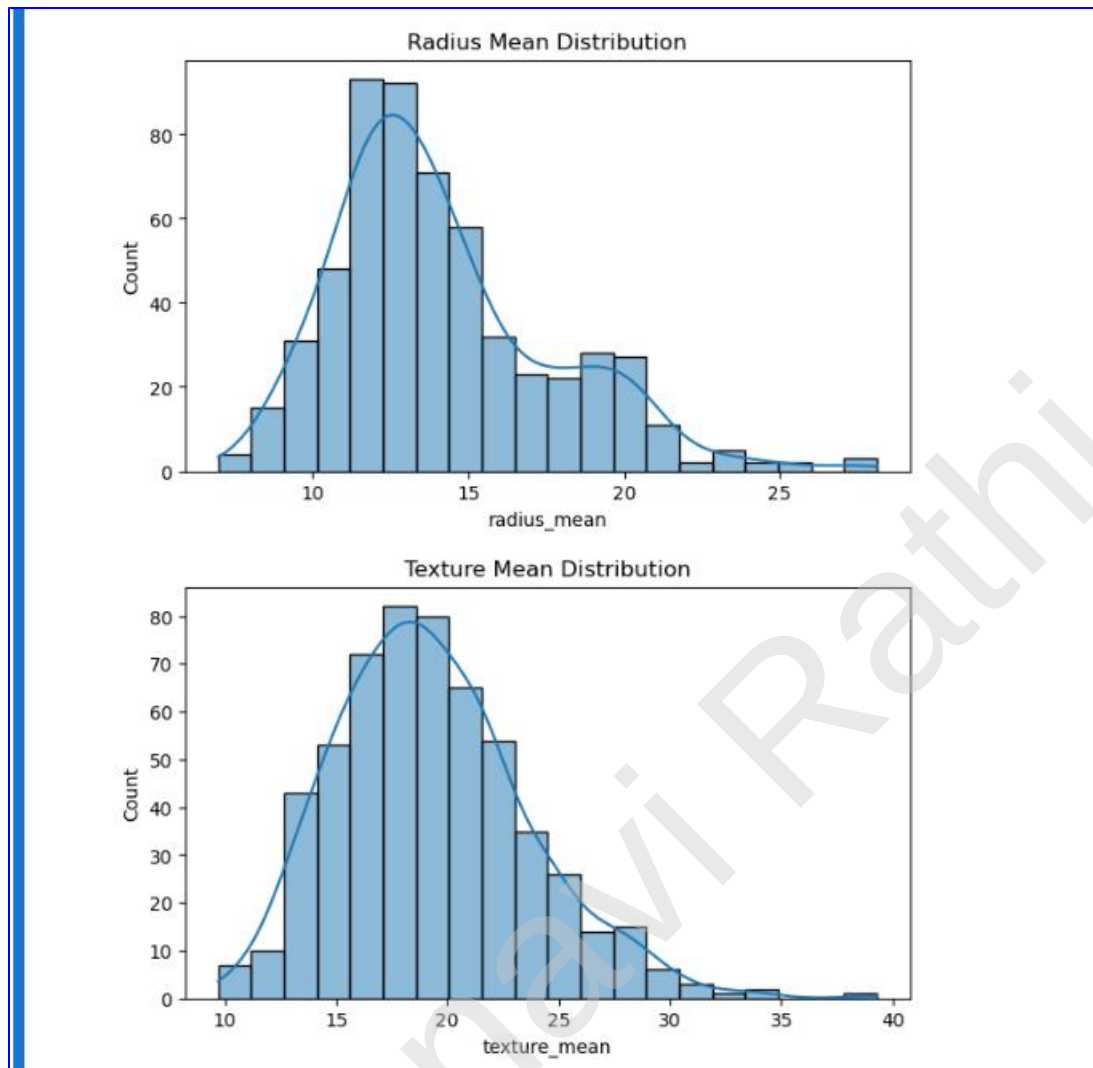
```
# Texture Mean Distribution
plt.figure(figsize=(7,4))
sns.histplot(df["texture_mean"], bins=20, kde=True)
plt.title("Texture Mean Distribution")
plt.show()

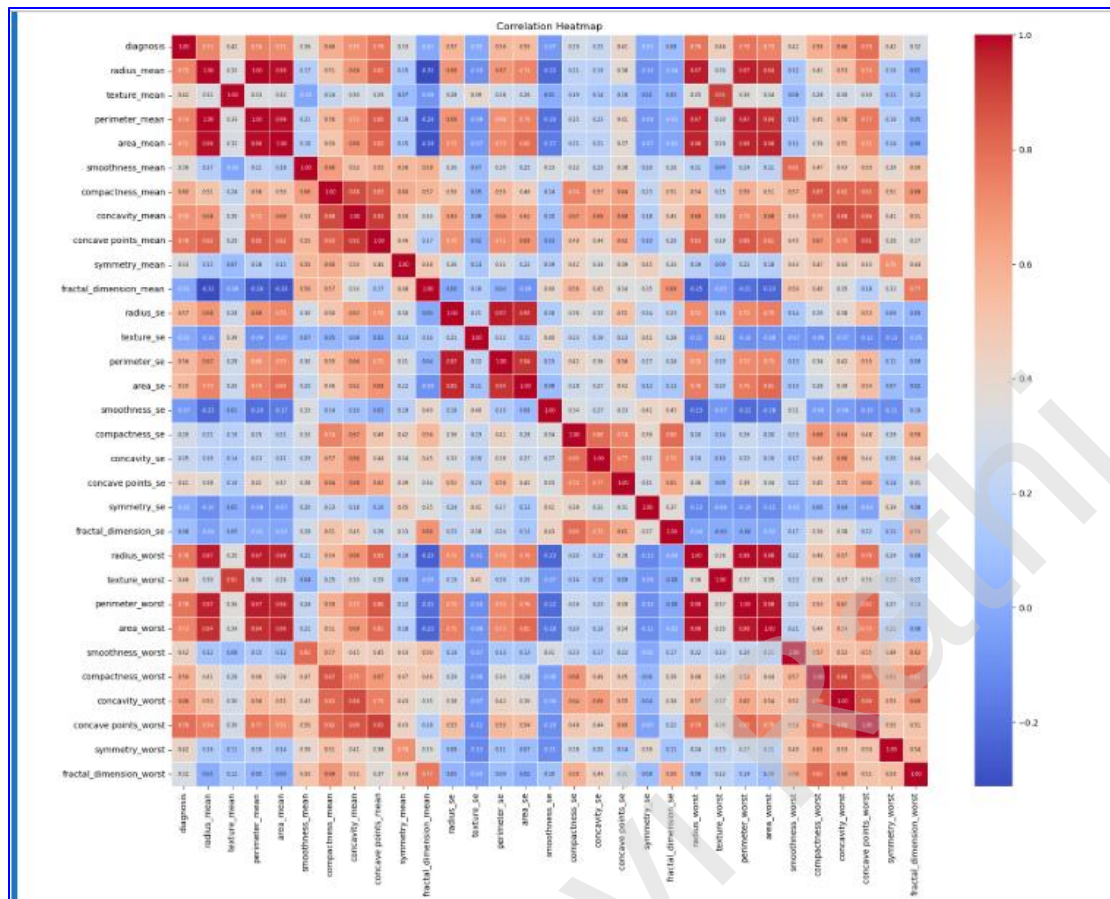
# Correlation Heatmap
plt.figure(figsize=(20, 16))

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    linewidths=0.5,
    annot_kws={"size": 6}
)

plt.title("Correlation Heatmap")
plt.xticks(rotation=90)
plt.yticks(rotation=0)
plt.show()
```







5) Feature Encoding & Scaling

```
[38]: '''
Feature Scaling
'''

from sklearn.preprocessing import StandardScaler

# Save column names
columns = X.columns

# Scale Features
scaler = StandardScaler()
X = scaler.fit_transform(X)

print("Feature Scaling Completed Successfully.")

Feature Scaling Completed Successfully.
```

6) Train-Test Split

```
[39]: '''  
Train-Test Split  
'''  
  
from sklearn.model_selection import train_test_split  
  
# Split dataset into training and testing sets  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42,  
    stratify=y  
)  
  
print("X_train Shape :", X_train.shape)  
print("X_test Shape :", X_test.shape)  
print("y_train Shape :", y_train.shape)  
print("y_test Shape :", y_test.shape)  
  
X_train Shape : (455, 30)  
X_test Shape : (114, 30)  
y_train Shape : (455,)  
y_test Shape : (114,)
```

Q2. Classification Algorithms

7) Logistic Regression Model Training

```
[33]: '''  
Logistic Regression Model Training  
'''  
  
from sklearn.linear_model import LogisticRegression  
  
# Create model  
lr_model = LogisticRegression(max_iter=1000)  
  
# Train model  
lr_model.fit(X_train, y_train)  
  
print("Logistic Regression model trained successfully.")  
  
Logistic Regression model trained successfully.
```

```
[34]: '''  
Model Prediction  
'''  
  
# Predict on test data  
y_pred_lr = lr_model.predict(X_test)  
  
print("First 10 Actual Values:")  
print(y_test.iloc[:10].values)  
  
print("\nFirst 10 Predicted Values:")  
print(y_pred_lr[:10])  
  
First 10 Actual Values:  
[0 1 0 1 0 0 1 0 0 0]  
  
First 10 Predicted Values:  
[0 1 0 1 1 0 1 0 0 0]
```

```
[35]: '''
Model Evaluation
'''

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

accuracy_lr = accuracy_score(y_test, y_pred_lr)

print("Accuracy :", round(accuracy_lr,4))

print("\nConfusion Matrix:\n")
print(confusion_matrix(y_test, y_pred_lr))

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_lr))
```

Accuracy : 0.9649

Confusion Matrix:

```
[[71  1]
 [ 3 39]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.96	0.99	0.97	72
1	0.97	0.93	0.95	42
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

8) Decision Tree Classifier Model Training

```
[36]: '''  
Decision Tree Classifier Model Training  
'''  
  
from sklearn.tree import DecisionTreeClassifier  
  
# Create model  
dt_model = DecisionTreeClassifier(  
    random_state=42  
)  
  
# Train model  
dt_model.fit(X_train, y_train)  
  
print("Decision Tree Model Trained Successfully.")  
  
Decision Tree Model Trained Successfully.
```

```
[37]: '''  
Model Prediction  
'''  
  
y_pred_dt = dt_model.predict(X_test)  
  
print("First 10 Actual Values:")  
print(y_test.iloc[:10].values)  
  
print("\nFirst 10 Predicted Values:")  
print(y_pred_dt[:10])  
  
First 10 Actual Values:  
[0 1 0 1 0 0 1 0 0 0]  
  
First 10 Predicted Values:  
[0 1 1 1 0 0 1 0 0 0]
```

```
[38]: '''
      Model Evaluation
      '''

      accuracy_dt = accuracy_score(y_test, y_pred_dt)

      print("Accuracy :", round(accuracy_dt, 4))

      print("\nConfusion Matrix:\n")
      print(confusion_matrix(y_test, y_pred_dt))

      print("\nClassification Report:\n")
      print(classification_report(y_test, y_pred_dt))
```

Accuracy : 0.9298

Confusion Matrix:

```
[[68  4]
 [ 4 38]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.94	0.94	0.94	72
1	0.90	0.90	0.90	42
accuracy			0.93	114
macro avg	0.92	0.92	0.92	114
weighted avg	0.93	0.93	0.93	114

9) Support Vector Machine (SVM) Model Training

```
Support Vector Machine (SVM) Model Training
'''

from sklearn.svm import SVC

# Create model
svm_model = SVC(
    kernel="rbf",
    random_state=42
)

# Train model
svm_model.fit(X_train, y_train)

print("SVM Model Trained Successfully.")

SVM Model Trained Successfully.

[40]: '''
Model Prediction
'''

y_pred_svm = svm_model.predict(X_test)

print("First 10 Actual Values:")
print(y_test.iloc[:10].values)

print("\nFirst 10 Predicted Values:")
print(y_pred_svm[:10])

First 10 Actual Values:
[0 1 0 1 0 0 1 0 0 0]

First 10 Predicted Values:
[0 1 0 0 0 0 1 0 0 0]
```



```
[41]: '''
      Model Evaluation
      '''

      accuracy_svm = accuracy_score(y_test, y_pred_svm)

      print("Accuracy :", round(accuracy_svm, 4))

      print("\nConfusion Matrix:\n")
      print(confusion_matrix(y_test, y_pred_svm))

      print("\nClassification Report:\n")
      print(classification_report(y_test, y_pred_svm))
```

Accuracy : 0.9737

Confusion Matrix:

```
[[72  0]
 [ 3 39]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.96	1.00	0.98	72
1	1.00	0.93	0.96	42
accuracy			0.97	114
macro avg	0.98	0.96	0.97	114
weighted avg	0.97	0.97	0.97	114

10) KNN Model Training

```
[42]: '''  
      KNN Model Training  
      '''  
  
      from sklearn.neighbors import KNeighborsClassifier  
  
      accuracy_scores = {}  
  
      for k in [3,5,7]:  
  
          knn = KNeighborsClassifier(n_neighbors=k)  
  
          knn.fit(X_train,y_train)  
  
          y_pred = knn.predict(X_test)  
  
          accuracy_scores[k] = accuracy_score(y_test,y_pred)  
  
          print(f"k = {k} --> Accuracy = {accuracy_scores[k]:.4f}")
```

```
k = 3 --> Accuracy = 0.9386  
k = 5 --> Accuracy = 0.9561  
k = 7 --> Accuracy = 0.9561
```

```
[43]: '''  
      Select Best K Value  
      '''  
  
      best_k = max(accuracy_scores,key=accuracy_scores.get)  
  
      print("Best K Value :",best_k)
```

```
Best K Value : 5
```

```
[44]: '''  
      Final KNN Model Training  
      '''  
  
      knn_model = KNeighborsClassifier(n_neighbors=best_k)  
  
      knn_model.fit(X_train,y_train)  
  
      print("Final KNN Model Trained Successfully.")
```

```
Final KNN Model Trained Successfully.
```

```
[45]: '''  
      Model Prediction  
      '''  
  
      y_pred_knn = knn_model.predict(X_test)  
  
      print("First 10 Actual Values:")  
      print(y_test.iloc[:10].values)  
  
      print("\nFirst 10 Predicted Values:")  
      print(y_pred_knn[:10])
```

```
First 10 Actual Values:  
[0 1 0 1 0 0 1 0 0 0]
```

```
First 10 Predicted Values:  
[0 1 0 0 0 0 1 0 0 0]
```

11) Naive Bayes Model Training

```
[46]: '''  
Naive Bayes Model Training  
'''  
  
from sklearn.naive_bayes import GaussianNB  
  
nb_model = GaussianNB()  
  
nb_model.fit(X_train,y_train)  
  
print("Naive Bayes Model Trained Successfully.")  
  
Naive Bayes Model Trained Successfully.
```

```
[47]: '''  
Model Prediction  
'''  
  
y_pred_nb = nb_model.predict(X_test)  
  
print("First 10 Actual Values:")  
print(y_test.iloc[:10].values)  
  
print("\nFirst 10 Predicted Values:")  
print(y_pred_nb[:10])  
  
First 10 Actual Values:  
[0 1 0 1 0 0 1 0 0 0]  
  
First 10 Predicted Values:  
[0 1 0 0 0 1 1 0 0 0]
```

```
[48]: '''  
Model Evaluation  
'''  
  
accuracy_nb = accuracy_score(y_test,y_pred_nb)  
  
print("Accuracy :",round(accuracy_nb,4))  
  
print("\nConfusion Matrix:\n")  
print(confusion_matrix(y_test,y_pred_nb))  
  
print("\nClassification Report:\n")  
print(classification_report(y_test,y_pred_nb))  
  
Accuracy : 0.9211  
  
Confusion Matrix:  
[[69  3]  
 [ 6 36]]  
  
Classification Report:  
  
              precision    recall  f1-score   support  
  
    0             0.92       0.96       0.94         72  
    1             0.92       0.86       0.89         42  
  
   accuracy              0.92         114  
  macro avg              0.92       0.91       0.91         114  
 weighted avg              0.92       0.92       0.92         114
```

Q4. Best Model Selection & Saving

12) Performance Comparison of Models

```
[49]: '''  
Performance Comparison of Models  
'''  
  
from sklearn.metrics import (  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score  
)  
  
# Create Comparison Table  
comparison = pd.DataFrame({  
    "Algorithm": [  
        "Logistic Regression",  
        "Decision Tree",  
        "Support Vector Machine",  
        "K-Nearest Neighbors",  
        "Naive Bayes"  
    ],  
    "Accuracy": [  
        accuracy_score(y_test, y_pred_lr),  
        accuracy_score(y_test, y_pred_dt),  
        accuracy_score(y_test, y_pred_svm),  
        accuracy_score(y_test, y_pred_knn),  
        accuracy_score(y_test, y_pred_nb)  
    ],  
    "Precision": [  
        precision_score(y_test, y_pred_lr),  
        precision_score(y_test, y_pred_dt),  
        precision_score(y_test, y_pred_svm),  
        precision_score(y_test, y_pred_knn),  
        precision_score(y_test, y_pred_nb)  
    ]  
})
```

```

    ],
    "Recall": [
        recall_score(y_test, y_pred_lr),
        recall_score(y_test, y_pred_dt),
        recall_score(y_test, y_pred_svm),
        recall_score(y_test, y_pred_knn),
        recall_score(y_test, y_pred_nb)
    ],
    "F1 Score": [
        f1_score(y_test, y_pred_lr),
        f1_score(y_test, y_pred_dt),
        f1_score(y_test, y_pred_svm),
        f1_score(y_test, y_pred_knn),
        f1_score(y_test, y_pred_nb)
    ]
}
})

comparison = comparison.round(4)

print("Model Performance Comparison:\n")
print(comparison)

```

Model Performance Comparison:

	Algorithm	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.9649	0.9750	0.9286	0.9512
1	Decision Tree	0.9298	0.9048	0.9048	0.9048
2	Support Vector Machine	0.9737	1.0000	0.9286	0.9630
3	K-Nearest Neighbors	0.9561	0.9744	0.9048	0.9383
4	Naive Bayes	0.9211	0.9231	0.8571	0.8889

```

[50]: '''
Best Performing Algorithm
'''

best_algorithm = comparison.loc[
    comparison["Accuracy"].idxmax(),
    "Algorithm"
]

print("Best Performing Algorithm:", best_algorithm)

print("\nConclusion:")
print(f"{best_algorithm} achieved the highest accuracy among all the classification models and is considered the best-

```

Best Performing Algorithm: Support Vector Machine

Conclusion:

Support Vector Machine achieved the highest accuracy among all the classification models and is considered the best-performing model for the Employee Attrition dataset.

```

[51]: '''
Best Classification Model
'''

best_classification_model = comparison.loc[
    comparison["Accuracy"].idxmax(),
    "Algorithm"
]

print("Best Classification Model:", best_classification_model)

```

Best Classification Model: Support Vector Machine

13) Save Best Classification Model

```
[52]: '''  
Save Best Classification Model  
'''  
  
import joblib  
  
if best_classification_model == "Logistic Regression":  
    best_model = lr_model  
  
elif best_classification_model == "Decision Tree":  
    best_model = dt_model  
  
elif best_classification_model == "Support Vector Machine":  
    best_model = svm_model  
  
elif "KNN" in best_classification_model:  
    best_model = knn_model  
  
else:  
    best_model = nb_model  
  
joblib.dump(best_model, "best_classification_model.pkl")  
joblib.dump(scaler, "classification_scaler.pkl")  
joblib.dump(columns, "classification_columns.pkl")  
  
print("Best Classification Model Saved Successfully.")  
  
Best Classification Model Saved Successfully.
```

Q5. Streamlit Multi-Model App

The screenshot displays the 'AI/ML Multi-Model Prediction System' interface. On the left, a sidebar titled 'Navigation' contains two dropdown menus: 'Select Problem Type' (set to 'Regression') and 'Select Algorithm' (set to 'Linear Regression'). The main area is titled 'Medical Insurance Cost Prediction' and prompts the user to 'Enter the patient details below.' The form includes input fields for Age (30), BMI (25.00), Number of Children (0), Gender (male), Smoker (yes), and Region (northeast). A 'Predict Insurance Charges' button is located below the form. The result is displayed as 'Predicted Insurance Charges: ₹ 27,297.43' in a green bar. The top right corner features 'Share', 'star', and 'refresh' icons, and the bottom right corner has a 'Manage app' link.

The screenshot displays the 'AI Medical Prediction System' interface. On the left, a sidebar titled 'Navigation' contains two dropdown menus: 'Select Problem Type' (set to 'Classification') and 'Select Algorithm' (set to 'Logistic Regression'). The main area is titled 'Predict Breast Cancer' and shows the result 'Prediction : Benign (No Cancer Detected)' in a green bar. Below this, a 'Prediction Confidence' section shows 'Benign : 50.97%' and 'Malignant : 49.03%' with a corresponding progress bar. The footer text reads '© AI Medical Prediction System', 'Built using Machine Learning Algorithms | Streamlit Deployment', and '© 2026 Vaishnavi Rathni'. The top right corner features 'Share', 'star', and 'refresh' icons, and the bottom right corner has a 'Manage app' link.

Please Check : <https://ai-prediction-systemapp-nxckwqvfx8rsi42u99nixk.streamlit.app/>